CHAPTER 1

Overview

1.1 Digital systems

Analog vs digital

Boolean algebra

Logic gates

Expressions & K-maps

1.2 SDLC

Waterfall case

Agile case

1.3 Networks

OSI model

TCP/IP

CLASS XI · COMPUTER SCIENCE

Unit 1: Computer Systems

Condensed lecture notes — digital logic, software life cycle, and network models

✓ Learning outcomes

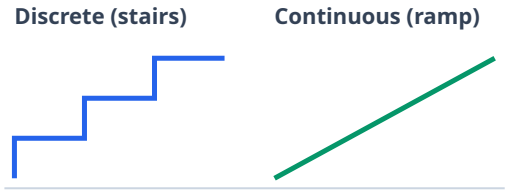
- Differentiate analog vs digital representation and build truth tables
- Apply Boolean operators and construct logic circuits (e.g. Logisim)
- Explain SDLC phases, Waterfall vs Agile, with case-study patterns
- Describe OSI (7 layers) and TCP/IP (4 layers) and compare them

1.1 Digital systems foundation

Discrete vs continuous

Discrete quantities are countable (students in a class, books on a shelf). Values jump in steps — like *stairs*.

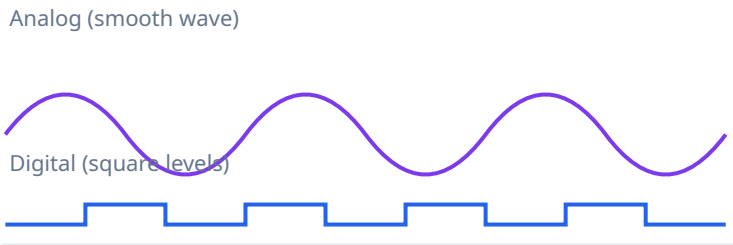
Continuous quantities vary smoothly in a range (temperature, height, time) — like a *ramp*.



Digital signals mirror discrete steps; analog mirrors continuous change.

Analog vs digital signals

Computers use **digital** data: voltage levels interpreted as binary **0** and **1** (OFF/ON). **Analog** signals vary continuously (e.g. old radio waves, thermometer mercury).



Sampling and quantization convert analog information into digital form.

1.1.4 Boolean algebra & operations

Boolean algebra uses two values: 0 (false) and 1 (true). Standard operators:

> **AND** ($\cdot$): $Y = A \cdot B$ — output 1 only if *all* inputs are 1.
> **OR** ($+$): $Y = A + B$ — output 1 if *any* input is 1.
> **NOT** ($'$): $Y = A'$ — unary; inverts the input.

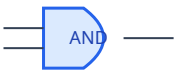
Precedence: NOT → AND → OR (use parentheses when in doubt).

Truth table pattern (2 inputs)

A	B	$A \cdot B$	$A + B$	$A \oplus B$ (XOR)
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

1.1.7 Logic gates

Gates implement Boolean functions in hardware. Bubble on output = inversion (NAND, NOR, XNOR).



AND — D-shape



OR — curved back



NOT — triangle + bubble



XOR — extra line on input side

Universal gates: NAND and NOR can implement any other gate. XOR outputs 1 when inputs differ; XNOR when they match.

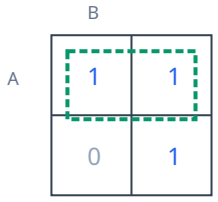
Example circuit: 2-input AND for $Y = A \cdot B$ — inputs A, B; single output Y (Logisim: place gate, input pins, output pin; wire orthogonally).

1.1.8 Expressions, truth tables & K-maps

From expression → fill truth table row by row. From truth table → sum of minterms (SOP) or product of maxterms (POS).

Minterm: AND of literals where function = 1. **Maxterm:** OR of literals where function = 0.

Karnaugh map (K-map): 2-variable grid for simplifying SOP; group adjacent 1s in powers of two; larger groups drop more variables.



2×2 K-map — group adjacent 1s

Lecture notes · Unit 1 only

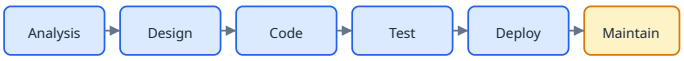
1.2 Software development life cycle (SDLC)

What is SDLC?

SDLC is a structured process for planning, building, testing, and maintaining software. It improves quality, cost control, and stakeholder alignment.

Typical phases

1. **Analysis** — gather requirements
2. **Design** — architecture, UI, database
3. **Implementation** — coding
4. **Testing** — verify against requirements
5. **Deployment** — release to users
6. **Maintenance** — fixes and enhancements



Linear phase flow (Waterfall-style)

Models at a glance

Aspect	Waterfall	Agile (Scrum)
Requirements	Fixed upfront	Evolving; user stories
Delivery	One release at end	Working software each sprint
Testing	Dedicated late phase	Continuous within sprints
Best when	Scope stable, regulated docs	Unclear scope, frequent feedback

1.2.5 Case study — Waterfall (library LMS)

Al-Noor Library Management System: fixed requirements, small team, 6-month plan — classic Waterfall fit.

- Testing (black box): e.g. search validation, automatic overdue alerts after fixes
- Deployment: pilot on librarian PC, data migration (~2,400 books), then school-wide
- Maintenance types: *corrective*, *adaptive*, *perfective*, *preventive* (backups)

1.2.6 Case study — Agile (mobile app)

Al-Noor Student Mobile App: unclear initial scope, multiple stakeholders, cross-platform (Flutter), 2-week sprints × 5.



Scrum cycle inside each sprint

Artifacts: **Product backlog** (prioritized user stories) → **Sprint backlog** → increment delivered for feedback.

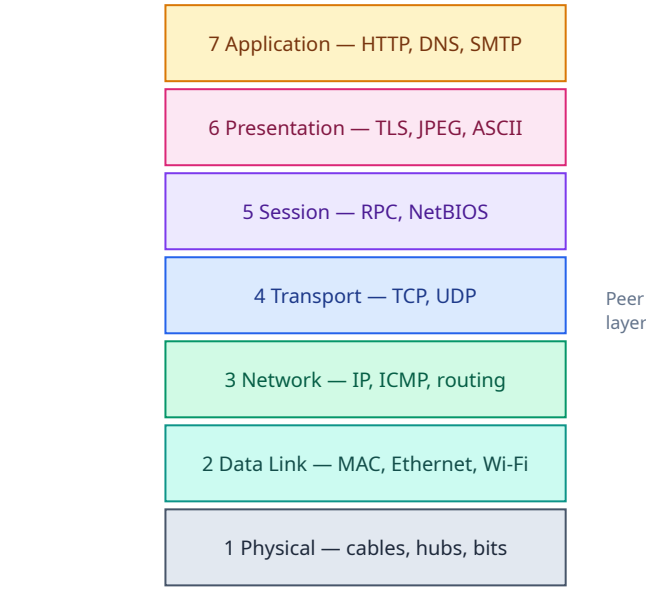
1.3 Communication models

A **communication model** defines layered rules so heterogeneous devices exchange data reliably — like a postal system with standardized envelopes and addresses.

- **OSI** — 7-layer reference model (teaching & design)
- **TCP/IP** — 4-layer practical stack used on the Internet

1.3.2 OSI seven-layer model

Mnemonic (L1→L7): “Please Do Not Throw Sausage Pizza Away” — Physical, Data Link, Network, Transport, Session, Presentation, Application.



Each layer serves the layer above and uses services of the layer below.

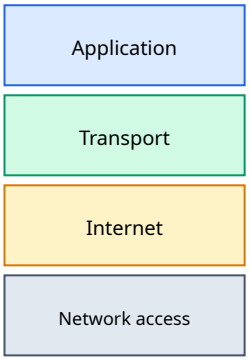
Encapsulation (sender → receiver)

Data moves **down** the stack on send (headers added = encapsulation), traverses the medium, then moves **up** on receive (de-encapsulation).



1.3.5 TCP/IP model

TCP/IP layer	Maps to OSI	Examples
Application	5–7	HTTP, FTP, DNS
Transport	4	TCP (reliable), UDP (fast)
Internet	3	IP, ICMP
Network access	1–2	Ethernet, Wi-Fi driver



TCP/IP — four layers on the real Internet

TCP — connection-oriented, retransmits lost segments, ordered delivery. **UDP** — connectionless, lower overhead (streaming, DNS queries).

Exam tip: OSI is the detailed reference; TCP/IP is the implementation stack. Know layer names, one protocol each, and direction of data flow.
